# Individual Claim Reserving

ACTL3143 & ACTL5111 Deep Learning for Actuaries

Patrick Laub

UNSW SYDNEY

> **Note**
>
> These slides are currently a work in progress. The full/final version will be completed soon.

# Lecture Outline

- **Introduction**
- An Individual Claim
- Benchmark: Aggregate Reserving
- Preprocessing One Claim
- Splitting Claims into Datasets
- Case Study: Synthetic Claims
- Wrapping Up

# Why is pricing different from reserving?

- *Pricing* is prospective and per-policy: no claim exists yet, and we predict the frequency/severity of hypothetical future claims from policy covariates (recall the French motor lectures).
- *Reserving* is about claims that have already happened: we condition on each claim's unfolding history — payments to date, case estimates — to predict what is still to be paid.
- Consequence for the ML setup: pricing rows are policies with static features; reserving rows are claim-quarter snapshots with *time-varying* features. That's why this lecture needs a whole preprocessing pipeline.

# From claim severity models to individual reserving

A claim severity model looks at a potential accident from the insured, and tries to work out from their covariates the likely size of such a claim.

An individual reserving model is more specific: an accident has occurred, the insured has been notified, and maybe parts of the claim have already been paid, now we need to guess how much remains in this realised claim.


This is potentially a more promising application for neural networks (more features!).

# Ultimate claim size

For claim $k$, we want to know the *ultimate claim size*

$$U_k := \sum_t Y_{k,t},$$

This is the sum of all *incremental payments* $Y_{k,t}$ (also called *partial payments*) over the complete lifetime of the claim.

**Assumption: payments are positive**

We assume $Y_{k,t} \geq 0$ throughout. In reality some payments are *negative* — called *recoveries* (e.g. money coming back from a third party or a salvaged vehicle). They are typically so rare and small relative to normal payments that ignoring them is reasonable (though this depends on the dataset).

# Outstanding claim amount

Say the valuation date is $v$, then we've seen the history of the claim $\mathcal{H}_{k,v}$ to that time, including past payments.

So the *outstanding* (future unpaid amount) for claim $k$ is

$$O_k(v) := \sum_{t>v} Y_{k,t}.$$

If the claim is still ongoing, then we have

$$U_k = \sum_{t \leq v} Y_{k,t} + O_k(v)\,.$$

In words:

$$\text{Ultimate} = \text{Cumulative payments before } v + \text{Outstanding amount from } v \text{ onwards}$$

# Total outstanding

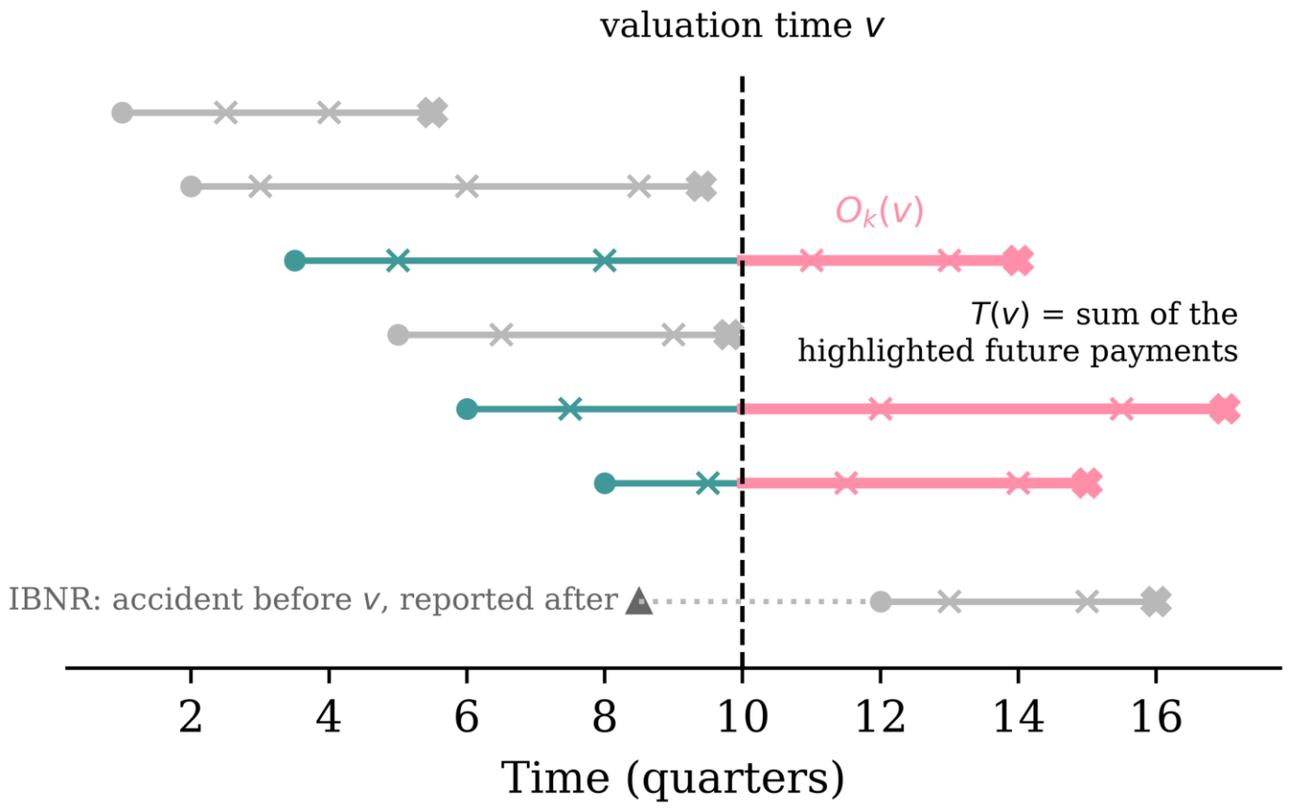
For a portfolio, we'd like to know the *total outstanding*

$$T(v) := \sum_{k \in \mathcal{O}(v)} O_k(v)\,,$$

where $\mathcal{O}(v)$ is the set of all open claims at valuation time $v$.

Note: this doesn't include *incurred but not reported (IBNR)* claims.

# The total outstanding, visually



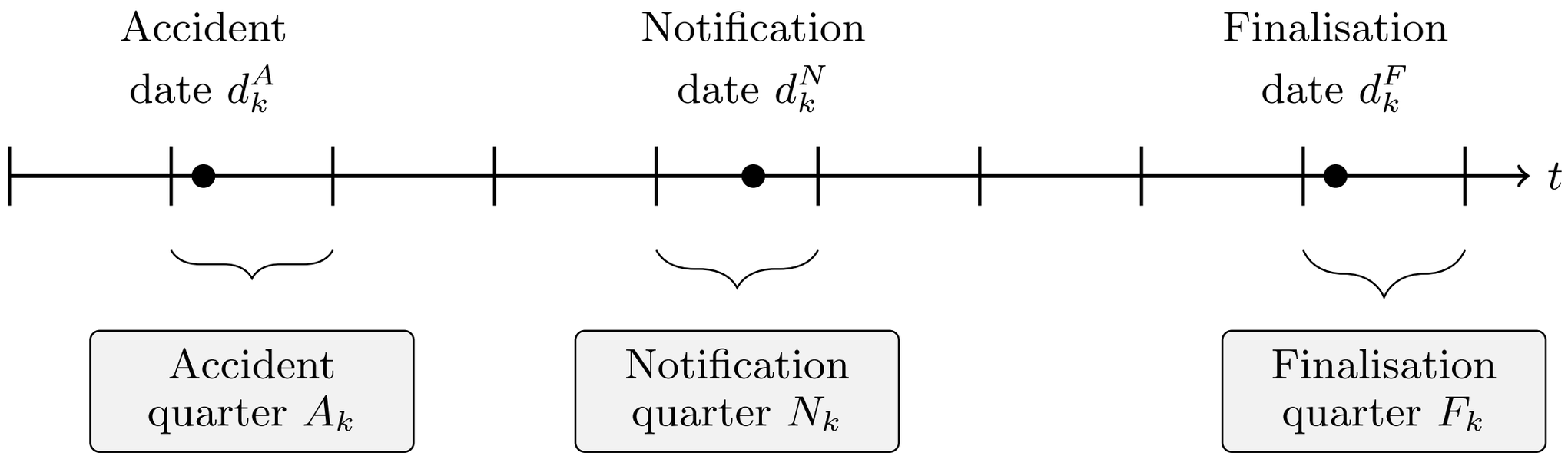

Each open claim's future payments (highlighted) get summed to give $T(v)$.

The bottom claim has *occurred* before $v$ but hasn't been *notified* yet. No individual model can reserve for a claim it cannot see — IBNR needs a separate adjustment.

# Lecture Outline

- Introduction
- **An Individual Claim**
- Benchmark: Aggregate Reserving
- Preprocessing One Claim
- Splitting Claims into Datasets
- Case Study: Synthetic Claims
- Wrapping Up

# The key dates




Key dates

A claim is *open* if not yet finalised.

$$\mathcal{O}(v) := \{k : d_k^F > v\}\,.$$

(Though, technically, claims sometimes finalise, and then get reopened with further payments, and refinalised again.)

# Series of incremental payments

| Event | Date | Amount |
|---|---|---|
| Accident | 2021-09-30 | |
| Notification | 2021-11-15 | |
| Payment #1 | 2022-05-17 | $9.00 |
| Payment #2 | 2022-08-31 | $2.00 |
| Payment #3 | 2022-10-23 | $2.00 |
| Payment #4 | 2022-12-20 | $4.00 |
| Payment #5 | 2023-02-19 | $2.00 |
| Payment #6 | 2023-05-31 | $3.00 |
| Payment #7 | 2023-07-08 | $18.00 |
| Payment #8 | 2023-09-10 | $8.00 |
| Finalisation | 2023-10-28 | |

# Case estimates

Alongside payments, the insurer holds a *case estimate* $C_{k,v} \geq 0$: a claims assessor's live, subjective estimate of the outstanding liability at the end of quarter $v$.

The assessor is aiming for

$$C_{k,v} \approx O_k(v)\,.$$

The crucial difference: $C_{k,v}$ is *known at time* $v$, whereas $O_k(v)$ is only knowable in retrospect, once the claim finalises.

So case estimates play two roles for us:

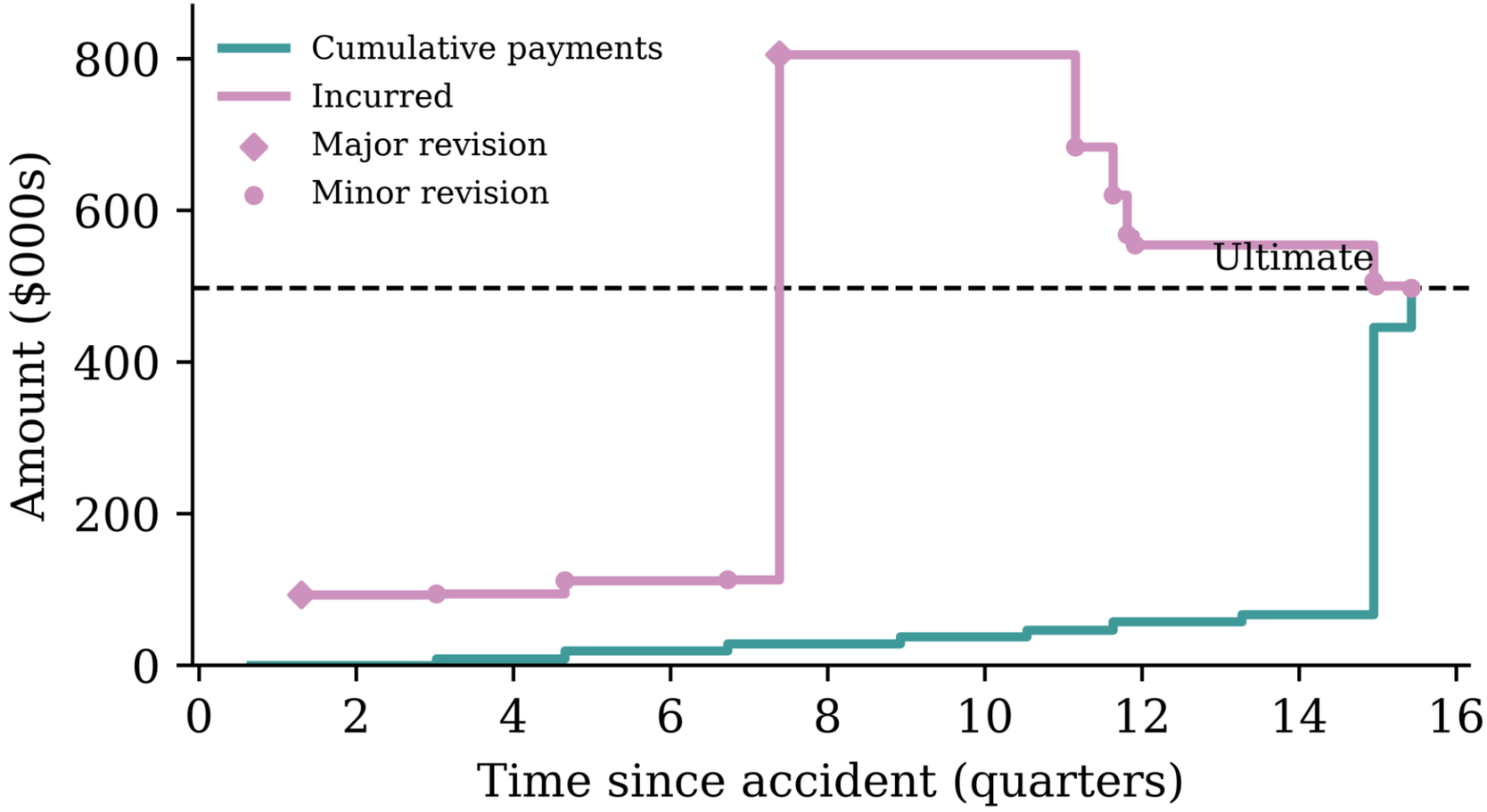
- a *strong predictor* to feed into the model, and
- a *benchmark to beat* (if the model can't out-predict the assessors, why bother?).

# Payments and case estimates together

The same running example claim, now with the assessor's revisions interleaved. Note the case estimate is a *level* (the current view of what's left), not a flow.

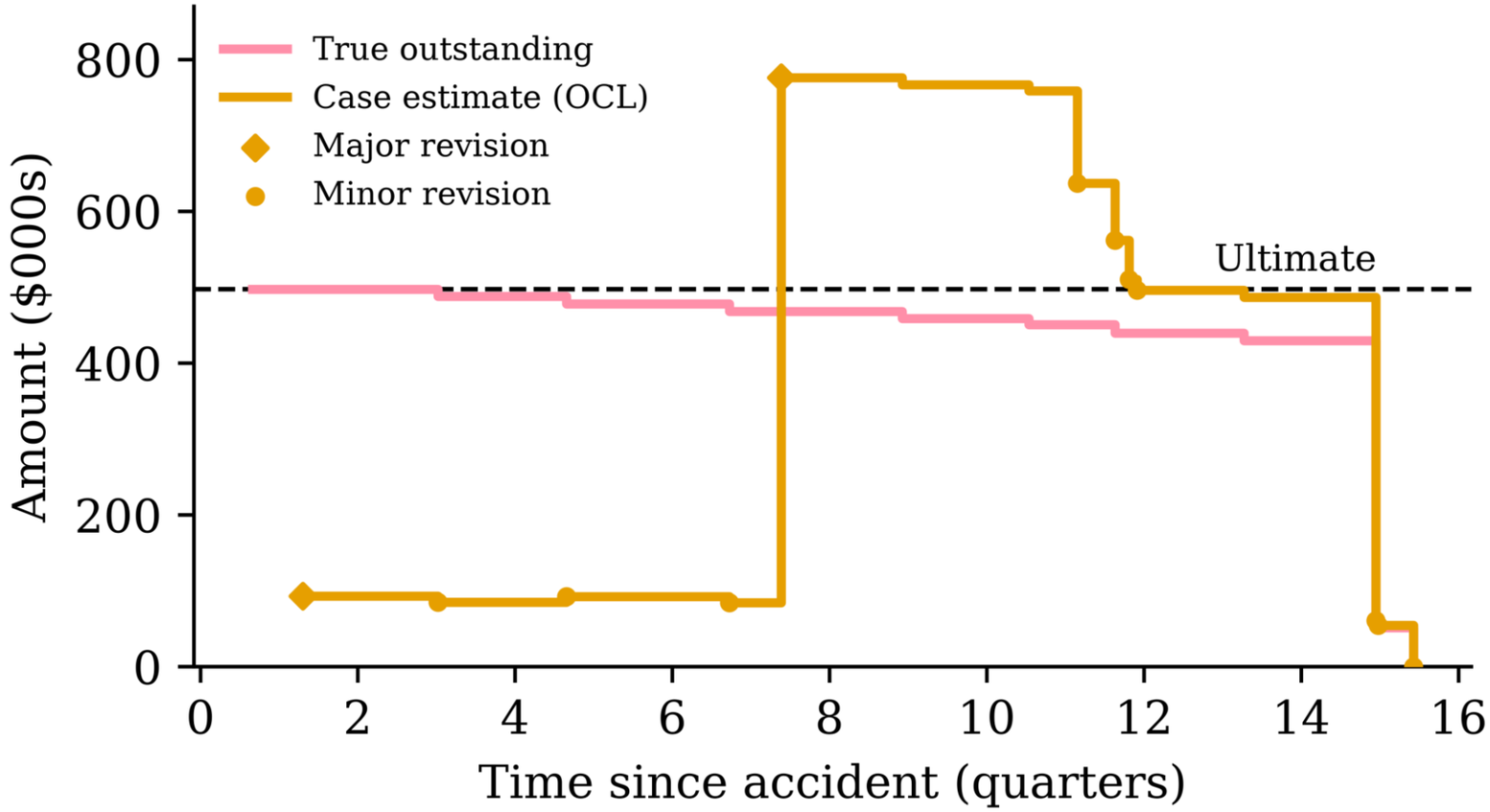
| Date | Event | Payment | Case estimate |
|---|---|---|---|
| 2021-11-15 | Notification, initial estimate | | $30.00 |
| 2022-05-17 | Payment #1 | $9.00 | $21.00 |
| 2022-06-20 | Review: worse than hoped | | $35.00 |
| 2022-08-31 | Payment #2 | $2.00 | $33.00 |
| 2022-10-23 | Payment #3 | $2.00 | $31.00 |
| 2022-12-20 | Payment #4 | $4.00 | $27.00 |
| 2023-02-19 | Payment #5 | $2.00 | $25.00 |
| 2023-05-31 | Payment #6 | $3.00 | $22.00 |
| 2023-07-08 | Payment #7 (settlement agreed) | $18.00 | $8.00 |
| 2023-09-10 | Payment #8 | $8.00 | $0.00 |
| 2023-10-28 | Finalisation | | $0.00 |

# Putting it all together: payments growing




Payments step up toward the ultimate. The *incurred*, which is cumulative payments plus the case estimate of the outstanding, is the assessor's running view of the ultimate.

# Putting it all together: outstanding shrinking




The same claim upside down: the true outstanding falls to zero as the payments arrive. The *outstanding claims liability (OCL)* is the assessor's live forecast of the outstanding, and drops to zero.

# Three levels of data visibility

1. *Individual transaction level*: Stores every transaction or other update for every claim. Highest granularity. Often just visible in the insured's bank statements, or in insurance company's finance records. Usually considered too granular to be useful for actuaries.
2. *Individual quarterly summaries*: This aggregates all the payments within a quarter (alternatively per month) for each claim. This is what the actuary would see.
3. *Portfolio quarterly summaries*: Aggregates all payments within a quarter (alternatively per month) across all claims. This feeds into a Chain Ladder style of reserving.

# Lecture Outline

- Introduction
- An Individual Claim
- **Benchmark: Aggregate Reserving**
- Preprocessing One Claim
- Splitting Claims into Datasets
- Case Study: Synthetic Claims
- Wrapping Up

# Aggregate reserving

You have seen the classical approach to reserving already. The insurer's past payments are aggregated into a *run-off triangle*: rows are accident years, columns are development years, and each cell holds the cumulative amount paid so far.

| Accident Year | Dev 1 | Dev 2 | Dev 3 |
|---|---|---|---|
| 2021 | 100 | 150 | 165 |
| 2022 | 120 | 180 | ? |
| 2023 | 140 | ? | ? |

The lower-right of the triangle is the future: it hasn't happened yet, and the *reserve* is our estimate of it.

# Chain ladder

Chain ladder fills in the triangle by assuming each accident year develops in the same proportions as the past ones.

- Development factor from Dev 1 to Dev 2: $f_1 = \frac{150+180}{100+120} = 1.5$
- Development factor from Dev 2 to Dev 3: $f_2 = \frac{165}{150} = 1.1$

Projecting forward:

| Accident Year | Dev 1 | Dev 2 | Dev 3 | Reserve |
|---|---|---|---|---|
| 2021 | 100 | 150 | 165 | 0 |
| 2022 | 120 | 180 | *198* | 18 |
| 2023 | 140 | *210* | *231* | 91 |

The total reserve here is $18 + 91 = 109$.

# What chain ladder ignores

Every claim in the same accident year is treated identically. The triangle has already thrown away:

- *who* was injured (age, injury severity),
- *how* the claim is progressing (legal representation, payment pattern),
- the case estimates set by claims assessors.

If two claims have paid out the same amount so far, chain ladder implicitly reserves the same for both — even if one is a minor injury about to settle and the other is heading to court.

# Why individual claim reserving?

With claim-level data and machine learning, we can use those discarded covariates to predict each claim's outstanding amount separately.

- The portfolio reserve is then just a sum: $T(v) = \sum_{k \in \mathcal{O}(v)} O_k(v)$.
- Per-claim predictions are useful beyond the total: claims triage (which claims need senior assessors?), segmenting the reserve by injury severity or legal representation, spotting claims whose case estimates look off.
- Chain ladder remains the *benchmark*: an individual model that can't beat the triangle isn't worth its complexity.

> **Note**
>
> Next week's guest lecture digs into *when and why* aggregate models break down, and the trade-offs between micro- and macro-level models. Today we focus on individual-level approach and how to implement it.

# Lecture Outline

- Introduction
- An Individual Claim
- Benchmark: Aggregate Reserving
- **Preprocessing One Claim**
- Splitting Claims into Datasets
- Case Study: Synthetic Claims
- Wrapping Up

# Noting the quarters

We work in quarters — the standard in actuarial practice. Each event date is binned into its calendar quarter, and we also track the *development quarter* (quarters since the accident).

| Event | Date | Calendar Quarter | Amount | Dev Quarter |
|---|---|---|---|---|
| Accident | 2021-09-30 | 2021Q3 | | Q1 |
| Notification | 2021-11-15 | 2021Q4 | | Q2 |
| Payment #1 | 2022-05-17 | 2022Q2 | $9.00 | Q4 |
| Payment #2 | 2022-08-31 | 2022Q3 | $2.00 | Q5 |
| Payment #3 | 2022-10-23 | 2022Q4 | $2.00 | Q6 |
| Payment #4 | 2022-12-20 | 2022Q4 | $4.00 | Q6 |
| Payment #5 | 2023-02-19 | 2023Q1 | $2.00 | Q7 |
| Payment #6 | 2023-05-31 | 2023Q2 | $3.00 | Q8 |
| Payment #7 | 2023-07-08 | 2023Q3 | $18.00 | Q9 |
| Payment #8 | 2023-09-10 | 2023Q3 | $8.00 | Q9 |

# Quarterly aggregation

All payments within a quarter collapse into one number, $Y_{k,t}$ — including quarters with *zero* payments, which are kept as rows.

| Quarter | Total Paid |
|---|---|
| 2022Q2 | $9.00 |
| 2022Q3 | $2.00 |
| 2022Q4 | $8.00 |
| 2023Q1 | $2.00 |
| 2023Q2 | $3.00 |
| 2023Q3 | $26.00 |

Case estimates within a quarter are summarised by their *latest* value (the estimate is a level, not a flow — summing revisions would be meaningless).

# Adjusting for inflation

A dollar paid in 2015 is not a dollar paid in 2025. If past payments enter the model in nominal terms, the model must waste capacity learning inflation — and drifts as soon as inflation changes.

- Deflate all dollar amounts to the valuation date using a suitable index (e.g. wage inflation for injury claims, since costs are dominated by wages and care).
- Fit the model in *real* dollars; re-inflate predictions afterwards if needed.

# From history to features

Everything known about claim $k$ at valuation quarter $v$:

$$\mathcal{H}_{k,v} := \{A_k, N_k, (Z_{k,t})_{t \le v},\ (Y_{k,t})_{t \le v},\ (C_{k,t})_{t \le v}\}$$

Some covariates $Z_{k,t}$ are *static* (age at accident, injury severity), others are *time-varying* (legal representation can switch on mid-claim).

Problem: this history has a *different length* for every $(k, v)$. Neural networks (the tabular kind we know) want a fixed-length vector.

The preprocessing pipeline is a deterministic map

$$X_{k,v} := \Phi(\mathcal{H}_{k,v}) \in \mathbb{R}^p.$$

# Two kinds of time-varying covariates

When $\Phi$ compresses the history, the time-varying covariates split into two kinds:

1. *Only the latest value matters.* E.g. legal representation status, or the current case estimate: keep $Z_{k,v}$, discard the path. (Implicitly a *Markov-style* assumption: the current state is a sufficient summary.)
2. *The path matters.* E.g. the payments-per-quarter series: a claim that paid \$40 in one early lump differs from one that dribbled out \$10 four times. These get summarised as functions of the whole history — they are *path-dependent* features.

Deciding which kind each covariate is *is itself a modelling choice*: treating the case estimate as "latest value only" throws away how often the assessor has revised it.

# Summarising the payment history

How does $\Phi$ compress a variable-length payment history into fixed columns? Summary statistics:

- development quarter $v - A_k$, notification delay $N_k - A_k$,
- count / sum / mean / std / min / max of past payments,
- the latest case estimate,
- current values of the time-varying covariates,
- the static covariates unchanged.

# Alternative: let an RNN read the history

The claim history is naturally a *sequence* — one vector per development quarter:

$$\left(Y_{k,t},\ C_{k,t},\ Z_{k,t}\right) \quad \text{for } t = N_k, \ldots, v.$$

So instead of hand-crafting summary statistics, we could (recall Week 4):

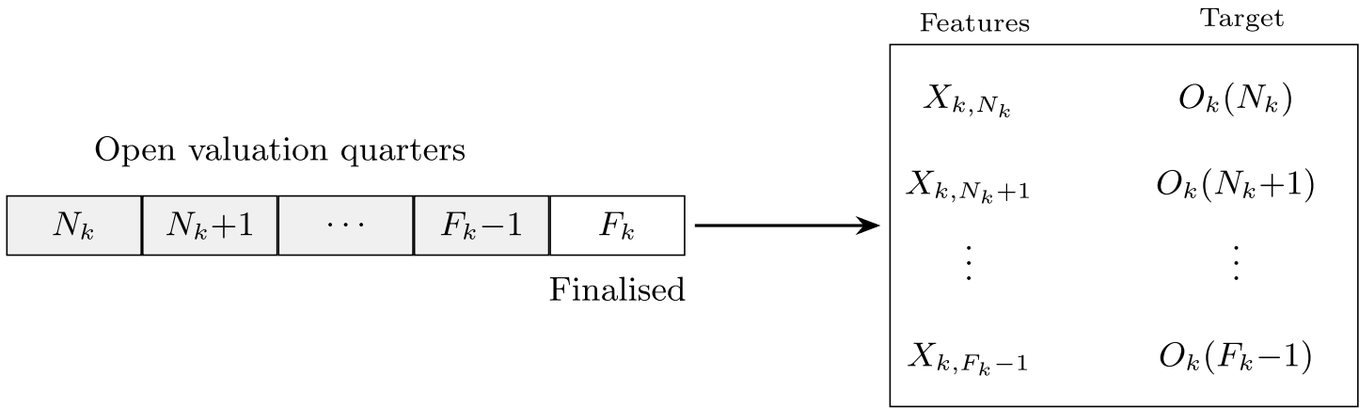
- feed the quarter-by-quarter vectors into a recurrent layer (e.g. GRU/LSTM),
- take the final hidden state as a *learned* summary of the history — a trainable replacement for $\Phi$'s count/sum/mean/etc.,
- concatenate the static covariates, and predict $O_k(v)$ with a dense head.

# Preparing features for a neural network

The second half of $\Phi$ is the standard tabular recipe from earlier weeks:

- `log1p` transform dollar amounts (heavy tails!),
- one-hot encode categoricals (age band, injury severity, legal representation),
- min–max scale numeric columns — with scalers *fit on the training set only*.

# Creating claim snapshots




One finalised claim, viewed retrospectively from *every* quarter it was open, yields one training row per quarter:

$$(X_{k,v},\ O_k(v)) \quad \text{for } v = N_k, \ldots, F_k - 1.$$

So a claim that took $F_k - N_k$ quarters to settle contributes $F_k - N_k$ rows. The full supervised dataset is

$$\mathcal{D} := \{(X_{k,v}, O_k(v)) : k \in \mathcal{I},\ v = N_k, \ldots, F_k - 1\}.$$

# Lecture Outline

- Introduction
- An Individual Claim
- Benchmark: Aggregate Reserving
- Preprocessing One Claim
- **Splitting Claims into Datasets**
- Case Study: Synthetic Claims
- Wrapping Up

# A portfolio of claims over time

Now zoom out from one claim to the whole portfolio. Every claim needs to be allocated to train, validation, or test — but claims *overlap in time*, so the familiar shuffled random split deserves suspicion.

# Two natural ways to split claims in time

Claims have two anchor dates you could split on:

- by *notification* quarter: a claim belongs to the era in which it *arrived*, or
- by *finalisation* quarter: a claim belongs to the era in which it *completed*.

Both are sensible-looking temporal splits (the interactive demo shows both). But they differ in one important way: under a notification split, the training set contains claims that are *still open* at the end of the training window — and for those, we don't know the true outstanding.

# The trouble with splitting by notification

For an in-progress claim, we haven't yet seen the target, we only know the *bound*:

$$U_k \geq \sum_{t \leq v} Y_{k,t} \, ,$$

i.e. the ultimate is at least what's been paid so far. This is a *censored* observation (the survival-analysis kind).

You could still train on these rows: add a loss term that penalises predictions of the ultimate *below* the payments made to date.

# It's simpler if we predict the outstanding

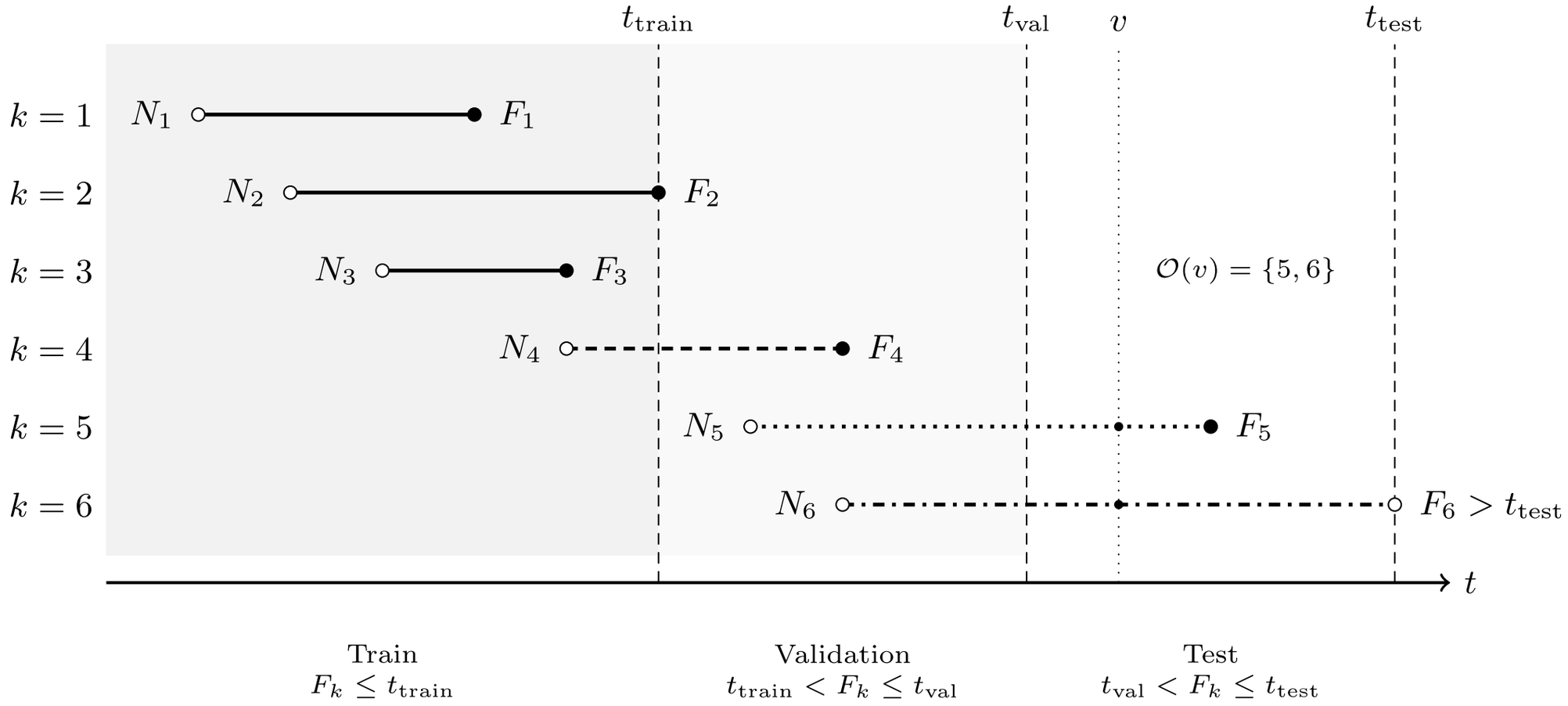
We chose to predict the *outstanding* $O_k(v)$ rather than the ultimate $U_k$ directly.

- For a censored row, the bound $U_k \geq$ paid so far translates to just $O_k(v) \geq 0$.
- Any sensible model of the outstanding would only predicts *positive* values.
- So the censored observations impose a constraint the model satisfies *by construction*, they carry no usable information.

So we predict the outstanding, split by *finalisation* quarter, and train only on fully-observed claims.

# Train / validation / test splits

Split claims by their *finalisation quarter*:




Those still open at $t_{\text{test}}$ are the *unfinalised* set.

# Why split by claim, not by row?

One claim generates $F_k - N_k$ snapshot rows, and consecutive snapshots of the same claim are *near-duplicates* (the history only grows by one quarter).

If we randomly shuffled *rows* into train/val/test, almost every test row would have a sibling in the training set. The model could just memorise claims — validation scores would look brilliant, deployment would disappoint.

So: *all snapshots from one claim go to the same set*. Splitting by finalisation quarter guarantees this, and keeps time flowing in one direction too.

# Censoring at the observation date

Claims still open at the end of the data window are *right-censored*: we can see their history, but not their target $O_k(v)$ (the future payments haven't happened yet).

- They can't be used for training or testing as their label is unknown.
- But they are precisely the claims we need predictions for in deployment! They form the *unfinalised set*.

# Lecture Outline

- Introduction
- An Individual Claim
- Benchmark: Aggregate Reserving
- Preprocessing One Claim
- Splitting Claims into Datasets
- **Case Study: Synthetic Claims**
- Wrapping Up

# Imports for the case study

The R side simulates the data and runs the classical benchmark:

```r
library(SPLICE)       # synthetic claims with case estimates
library(ChainLadder)  # the classical benchmark
library(arrow)        # parquet files: handing data from R to Python
```

The Python side does the machine learning:

```python
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import Ridge
from sklearn.metrics import root_mean_squared_error
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import (FunctionTransformer, MinMaxScaler,
                                   OneHotEncoder)
```

# Generating synthetic claims with SPLICE

SynthETIC (Avanzi et al., 2021) generates the claims and payments; SPLICE (Avanzi et al., 2023) adds the case estimates; the claim-level covariates plug in via SynthETIC's covariates module.

```
sim ← generate_data(
  n_claims_per_period = 100,   # ~4,000 claims in total
  n_periods           = 40,    # 40 quarters = 10 accident years
  complexity          = 5,     # the most realistic scenario
  random_seed         = 20260712,
  covariates_obj      = SynthETIC::test_covariates_obj
)
claims ← cbind(sim$claim_dataset, sim$covariates_data$data)
write_parquet(claims, "data/splice/claims.parquet")
write_parquet(sim$payment_dataset, "data/splice/payments.parquet")
write_parquet(sim$incurred_dataset, "data/splice/estimates.parquet")
```

Three tables: one row per *claim*, per *payment*, and per *case estimate transaction*.

# One synthetic claim's payments

One claim's payments (note the paired nominal/deflated columns):

```
payments[payments["claim_no"] == 1][["pmt_no", "payment_time", "payment_period", "payment
```

| | pmt_no | payment_time | payment_period | payment_size | payment_inflated |
|---|---|---|---|---|---|
| **0** | 1.0 | 4.226871 | 5.0 | 1977.969824 | 2577.772115 |
| **1** | 2.0 | 4.851022 | 5.0 | 1991.849601 | 2699.394047 |
| **2** | 3.0 | 5.683277 | 6.0 | 2312.694383 | 3301.991516 |
| **3** | 4.0 | 6.431716 | 7.0 | 16329.733127 | 24434.524512 |
| **4** | 5.0 | 6.961067 | 7.0 | 2120.736589 | 3280.323306 |

# One synthetic claim's case estimates

Its case estimate transactions (`OCL` is the case estimate):

```
estimates[estimates["claim_no"] == 1][["txn_time", "txn_type", "cumpaid", "OCL"]]
```

| | txn_time | txn_type | cumpaid | OCL |
|---|---|---|---|---|
| **0** | 3.646604 | Ma | 0.000000 | 35934.023245 |
| **1** | 4.226871 | P | 2577.772115 | 33356.251130 |
| **2** | 4.851022 | P | 5277.166162 | 30656.857083 |
| **3** | 5.683277 | P | 8579.157678 | 27354.865567 |
| **4** | 6.431716 | PMi | 33013.682190 | 3087.435635 |
| **5** | 6.961067 | PMi | 36294.005496 | 0.000000 |

# From transactions to snapshot rows

Bin each claim's key dates into quarters, and sort the claims into groups:

```
T_END = 40  # the valuation quarter (end of the observation window)

claims["notif_qtr"] = np.ceil(claims["occurrence_time"] + claims["notidel"]).astype(int)
claims["final_qtr"] = np.ceil(claims["occurrence_time"] + claims["notidel"] + claims["set

ibnr = claims["notif_qtr"] > T_END
unfinalised = ~ibnr & (claims["final_qtr"] > T_END)
flash = ~ibnr & ~unfinalised & (claims["final_qtr"] == claims["notif_qtr"])
observed = claims[~ibnr & ~unfinalised & ~flash]
```

```
4,027 claims: 2,726 finalised & usable, 883 unfinalised, 245 IBNR, 173 settled within a
quarter
```

Every category from the theory shows up in the counts.

# Building the snapshot grid

One row per claim per open quarter, with the quarterly payments $Y_{k,t}$ merged in (missing quarters become zero-payment rows):

```
n_snapshots = (observed["final_qtr"] - observed["notif_qtr"]).to_numpy()
grid = observed.loc[observed.index.repeat(n_snapshots),
    ["claim_no", "occurrence_period", "notif_qtr", "final_qtr", "notidel",
     "Legal Representation", "Injury Severity", "Age of Claimant"]].copy()
grid["quarter"] = grid["notif_qtr"] + grid.groupby("claim_no").cumcount()

quarterly = (payments.groupby(["claim_no", "payment_period"])["payment_size"]
             .sum().rename("Incremental").reset_index())
grid = grid.merge(quarterly, left_on=["claim_no", "quarter"],
                  right_on=["claim_no", "payment_period"], how="left")
grid["Incremental"] = grid["Incremental"].fillna(0.0)
grid = grid.drop(columns="payment_period")
```

# History summaries, case estimates, and the target

The path-dependent features (expanding summaries of the payment history), the current-state features (latest case estimate, forward-filled), and the target:

```python
g = grid.groupby("claim_no")["Incremental"]
grid["count_Incremental"] = ((grid["Incremental"] > 0)
                             .groupby(grid["claim_no"]).cumsum())
grid["sum_Incremental"] = g.cumsum()
for stat in ["mean", "std", "min", "max"]:
    grid[f"{stat}_Incremental"] = getattr(g.expanding(), stat)().to_numpy()
grid["std_Incremental"] = grid["std_Incremental"].fillna(0.0)

ultimate = payments.groupby("claim_no")["payment_size"].sum()
grid["Outstanding"] = grid["claim_no"].map(ultimate) - grid["sum_Incremental"]
grid["development_period"] = grid["quarter"] - grid["occurrence_period"] + 1
```

```python
estimates["txn_qtr"] = np.ceil(estimates["txn_time"]).astype(int)
latest = (estimates.sort_values("txn_time")
          .groupby(["claim_no", "txn_qtr"])["OCL"].last()
          .rename("Estimate").reset_index())
grid = grid.merge(latest, left_on=["claim_no", "quarter"],
                  right_on=["claim_no", "txn_qtr"], how="left")
grid["Estimate"] = grid.groupby("claim_no")["Estimate"].ffill()
grid = grid.drop(columns="txn_qtr")
```

# The dataset after preprocessing

```
grid[["claim_no", "quarter", "development_period", "Incremental",
      "sum_Incremental", "Estimate", "Outstanding"]].head(8)
```

| | claim_no | quarter | development_period | Incremental | sum_Incremental | |
|---|---|---|---|---|---|---|
| **0** | 1 | 4 | 4.0 | 0.000000 | 0.000000 | 359 |
| **1** | 1 | 5 | 5.0 | 3969.819426 | 3969.819426 | 306 |
| **2** | 1 | 6 | 6.0 | 2312.694383 | 6282.513808 | 273 |
| **...** | ... | ... | ... | ... | ... | ... |
| **5** | 2 | 3 | 3.0 | 0.000000 | 0.000000 | 157 |
| **6** | 2 | 4 | 4.0 | 0.000000 | 0.000000 | 157 |
| **7** | 2 | 5 | 5.0 | 10058.311380 | 10058.311380 | 541 |

8 rows × 7 columns

# Splitting by finalisation quarter

```python
T_TRAIN, T_VAL = 28, 34
splits = {
    "train": grid[grid["final_qtr"] ⩽ T_TRAIN],
    "val": grid[(grid["final_qtr"] > T_TRAIN) & (grid["final_qtr"] ⩽ T_VAL)],
    "test": grid[grid["final_qtr"] > T_VAL],
}
```

```
train    9,499 rows  1,575 claims  (finalisation quarters 2..28)
val      5,491 rows    609 claims  (finalisation quarters 29..34)
test     4,922 rows    542 claims  (finalisation quarters 35..40)
```

Note rows $\gg$ claims: each claim contributes one row per quarter it was open — and every one of a claim's rows lands in the same set.

# Benchmark: chain ladder

```r
payments ← read_parquet("data/splice/payments.parquet")
observed ← subset(payments, payment_period ⩽ 40)
observed$acc_year ← ceiling(observed$occurrence_period / 4)
observed$dev_year ← ceiling(observed$payment_period / 4) -
  observed$acc_year + 1

incr ← aggregate(payment_size ~ acc_year + dev_year, observed, sum)
tri ← incr2cum(as.triangle(incr, origin = "acc_year",
                           dev = "dev_year", value = "payment_size"))
round(tri / 1e6, 2)  # cumulative payments, $m
```

```
        dev_year
acc_year    1     2     3     4     5     6     7     8     9    10
       1  1.44 10.31 20.94 31.91 39.89 48.66 55.51 60.24 64.77 65.71
       2  1.36  9.72 23.94 31.12 38.18 51.38 59.53 61.35 63.60    NA
       3  1.16  8.90 24.57 34.37 40.07 47.59 53.12 53.95    NA    NA
       4  1.70 10.42 21.21 35.53 50.53 62.02 66.14    NA    NA    NA
       5  1.18  9.21 22.12 32.27 37.72 44.55    NA    NA    NA    NA
       6  1.16 12.26 26.07 36.08 46.09    NA    NA    NA    NA    NA
       7  2.99 14.13 28.93 36.01    NA    NA    NA    NA    NA    NA
       8  0.93  7.18 17.98    NA    NA    NA    NA    NA    NA    NA
       9  1.01  8.71    NA    NA    NA    NA    NA    NA    NA    NA
       10 1.20    NA    NA    NA    NA    NA    NA    NA    NA    NA
```

# The chain ladder reserve

```r
mack ← MackChainLadder(tri, est.sigma = "Mack")
reserve_cl ← summary(mack)$Totals["IBNR:", ]
true_outstanding ← sum(subset(payments, payment_period > 40)$payment_size)

cat(sprintf("Chain ladder reserve:   $%.1fm\n", reserve_cl / 1e6),
    sprintf("True total outstanding: $%.1fm\n", true_outstanding / 1e6))
```

```
Chain ladder reserve:   $208.9m
 True total outstanding: $264.6m
```

As this is synthetic data, we know the future and can compare against the true value.

# A baseline model: GLM

Before any neural network: ridge regression on the log outstanding, with the standard tabular recipe as an sklearn pipeline.

```python
dollar_cols = ["Incremental", "sum_Incremental", "mean_Incremental",
               "std_Incremental", "min_Incremental", "max_Incremental"]
numeric_cols = ["development_period", "notidel", "count_Incremental"]
cat_cols = ["Legal Representation", "Injury Severity", "Age of Claimant"]

def make_glm(with_estimate):
    dollars = dollar_cols + (["Estimate"] if with_estimate else [])
    preprocess = ColumnTransformer([
        ("dollars", make_pipeline(FunctionTransformer(np.log1p),
                                  MinMaxScaler()), dollars),
        ("numeric", MinMaxScaler(), numeric_cols),
        ("categorical", OneHotEncoder(drop="first"), cat_cols),
    ])
    return make_pipeline(preprocess, Ridge(alpha=1e-4))
```

Two versions: *without* and *with* the case estimate as a feature.

# How do the GLMs do?

```
X_train, X_test = splits["train"], splits["test"]
y_train = np.log1p(X_train["Outstanding"])
y_test = np.log1p(X_test["Outstanding"])

for with_est in [False, True]:
    glm = make_glm(with_est).fit(X_train, y_train)
    rmse = root_mean_squared_error(y_test, glm.predict(X_test))
    print(f"GLM {'with' if with_est else 'without'} case estimates:"
          f" test RMSE (log scale) = {rmse:.3f}")
```

```
GLM without case estimates: test RMSE (log scale) = 1.308
GLM with case estimates: test RMSE (log scale) = 1.327
```

```
rmse_ce = root_mean_squared_error(y_test, np.log1p(X_test["Estimate"]))
print(f"Case estimate alone: test RMSE (log scale) = {rmse_ce:.3f}")
```

```
Case estimate alone: test RMSE (log scale) = 1.748
```

Two questions for the printed numbers: does the model beat the assessors' estimates? Does giving it the case estimates help?

# Fitting a neural network

# Evaluating the models

# Lecture Outline

- Introduction
- An Individual Claim
- Benchmark: Aggregate Reserving
- Preprocessing One Claim
- Splitting Claims into Datasets
- Case Study: Synthetic Claims
- **Wrapping Up**

# Individual vs aggregate reserving

- *Aggregate* (chain ladder on triangles): simple, stable, industry standard; but throws away claim-level covariates and gives no per-claim view.
- *Individual*: predicts $O_k(v)$ per open claim; the portfolio reserve is just the sum $T(v)$; supports triage and segmentation; per-claim errors partially cancel in the aggregate.
- Individual level reserving is a data engineering headache, and the total still misses IBNR, so an individual model alone is *not yet* a full portfolio reserve.

# What we didn't cover

One slide, words only — pointers to where this goes next:

- *IBNR*: our model only reserves claims it can see; incurred-but-not-reported claims need a separate (usually aggregate) adjustment.
- *Uncertainty*: a reserve needs a risk margin, not just a point estimate; distributional forecasts and ensembling give prediction intervals.
- *When does granular beat aggregate?*

# References

Avanzi, B., Taylor, G., & Wang, M. (2023). SPLICE: A synthetic paid loss and incurred cost experience simulator. *Annals of Actuarial Science*, *17*(1), 7–35. https://doi.org/10.1017/S1748499522000057

Avanzi, B., Taylor, G., Wang, M., & Wong, B. (2021). SynthETIC: An individual insurance claim simulator with feature control. *Insurance: Mathematics and Economics*, *100*, 296–308. https://doi.org/10.1016/j.insmatheco.2021.06.004

UNSW SYDNEY